



عنوان پروژه:

طراحی و پیاده‌سازی یک پایشگر هوشمند ETL مبتنی بر LLM برای پاکسازی و یکپارچه‌سازی داده‌های جدولی

نام دانشجو : مهسا محمدی

نام استاد: سرکار خانم دکتر طاهائی

نیمسال تحصیلی ۱۴۰۴-۰۵

چکیده

مشکلات کیفیت داده یکی از چالش‌های اساسی در فرایندهای ETL محسوب می‌شوند و می‌توانند به کاهش دقت سیستم‌های تحلیلی و یادگیری ماشین منجر شوند. در بسیاری از کاربردهای واقعی، داده‌ها شامل خطاهایی مانند مقادیر گم‌شده، رکوردهای تکراری و ناسازگاری در ساختار هستند. روش‌های سنتی پاکسازی داده معمولاً مبتنی بر قوانین (Rule-Based) ثابت هستند و توانایی محدودی در درک معنایی داده‌ها دارند.

در سال‌های اخیر، مدل‌های زبانی بزرگ (LLMs) به دلیل توانایی درک معنایی و استدلال متنی، در حوزه‌های مختلفی از جمله آماده‌سازی داده، پاکسازی و یکپارچه‌سازی داده مورد توجه قرار گرفته‌اند. [1]

در این پروژه، یک چارچوب سبک برای پایش و پاکسازی هوشمند داده‌های ETL ارائه می‌شود که در آن از ترکیب روش‌های مبتنی بر قوانین و مدل‌های زبانی بزرگ به صورت محلی استفاده شده است. هدف اصلی سیستم، شناسایی و اصلاح خطاهای ارزیابی سیستم روی دیتاست‌های دانشگاهی معتبر (Adult Income، Titanic، OpenML) انجام می‌شود و با معیارهای استاندارد Completeness، Consistency، Duplicate Rate و Error Correction Rate سنجیده خواهد شد.

1. مقدمه

افزایش حجم داده‌های تولیدشده در سیستم‌های مدرن، کیفیت داده به یکی از مهم‌ترین عوامل در موفقیت سیستم‌های داده‌محور تبدیل شده است. بسیاری از داده‌های واقعی شامل خطاهایی مانند داده‌های گم‌شده، ناسازگاری فرمت‌ها و اختلاف در ساختار جداول هستند که می‌توانند عملکرد مدل‌های یادگیری ماشین را تحت تأثیر قرار دهند. [3]

فرایند ETL به عنوان یکی از اجزای اصلی مهندسی داده وظیفه استخراج، تبدیل و یکپارچه‌سازی داده‌ها را بر عهده دارد. با این حال، سیستم‌های سنتی ETL عمدتاً مبتنی بر قوانین از پیش تعریف‌شده هستند و توانایی محدودی در درک معنایی داده‌ها دارند. [4] در مقابل، مدل‌های زبانی بزرگ (LLMs) توانسته‌اند قابلیت‌های قابل توجهی در درک متن، استدلال معنایی و انجام عملیات هوشمند بر روی داده‌ها نشان دهند. پژوهش‌ها نشان می‌دهند که LLM ها می‌توانند در وظایفی مانند data cleaning، schema matching و data integration عملکرد مناسبی داشته باشند. [1], [5]

در این پروژه، از مدل‌های زبانی بزرگ استفاده می‌شود تا بدون وابستگی به سرویس‌های خارجی، قابلیت تحلیل و پاکسازی معنایی داده‌ها فراهم شود. با این وجود، بیشتر پژوهش‌های موجود تنها بر عملیات پاکسازی یا تولید workflow تمرکز دارند و کمتر به موضوعاتی مانند مانیتورینگ ETL، لاگ‌گیری و مشاهده‌پذیری در چارچوب MLOps پرداخته‌اند.

2. بیان مسئله

2.1 چرا این مسئله مهم است؟

در سیستم‌های داده محور واقعی، کیفیت پایین داده می‌تواند به خرابی مدل‌های یادگیری ماشین، خطاهای تحلیلی پرهزینه و تصمیمات نادرست سازمانی منجر شود. تحقیقات نشان داده‌اند که بین ۶۰ تا ۸۰ درصد زمان یک پروژه داده صرف آماده‌سازی و پاکسازی داده می‌شود [4]. با این حال، اکثر ابزارهای موجود ETL فاقد قابلیت‌های معنایی و هوشمند هستند.

از منظر کاربردی، شکاف‌های زیر در سیستم‌های موجود مشهود است:

- تنها خطاهای ساختاری را می‌یابند و ابزارهای Rule-Based مانند Great Expectations توانایی تشخیص ناسازگاری‌های معنایی را ندارند.
- راحل‌های مبتنی بر LLM ابری مانند ChatGPT API برای داده‌های حساس سازمانی قابل استفاده نیستند.
- هیچ ابزار یکپارچه‌ای که هم پاکسازی هوشمند و هم مانیتورینگ را در یک چهارچوب MLOps ارائه دهد، در مقیاس دانشگاهی وجود ندارد.

2.2 چرا این مسئله از دید MLOps مهم است؟

MLOps به عنوان یک حوزه، تمرکز خود را در اتوماسیون، مشاهده‌پذیری و چرخه حیات مدل‌ها قرار داده است. [9]

اما یکی از خلاءهای جدی در ادبیات MLOps غفلت از لایه داده به عنوان بخش بالادستی، پایپ‌لاین است. این پروژه استدلال می‌کند که:

- پایش کیفیت داده در زمان اجرا (Runtime Data Quality Monitoring) جزء جدایی ناپذیر یک سیستم MLOps سالم است.
- Drift در داده که اغلب ریشه در تغییر و توزیع ورودی‌های ETL دارد، باید در همان لایه ETL شناسایی شود و نه بعد از استقرار مدل.
- لاگ‌گیری ساختارمند در ETL، قابلیت Reproducibility و Auditability ضروری برای یک خط لوله MLOps را فراهم می‌کند.

2.3 سناریو کاربردی

فرض کنید یک تیم دانشگاهی یا یک استارت‌آپ کوچک در حال ساخت یک سیستم پیش‌بینی بر اساس داده‌های جدولی است. آن‌ها با چالش‌های زیر مواجه‌اند:

- داده‌های ورودی از چند منبع ناهمگن جمع‌آوری می‌شوند.
 - ابری گران است API بودجه و زیرساخت محدود است و استفاده از
 - تیم نمی‌تواند هر روز دستی کیفیت داده را بررسی کند
- این پروژه برای این سناریو یک راحل اتوماتیک، محلی و قابل مشاهده ارائه می‌دهد.

3. اهداف پروژه

3.1 هدف اصلی

هدف اصلی این پژوهش، طراحی و پیاده‌سازی یک چارچوب یکپارچه برای پایش هوشمند فرایندهای ETL است که با بهرگیری همزمان از روش‌های مبتنی بر قواعد (Rule-Based) و مدل‌های زبانی بزرگ محلی (Local LLM)، کیفیت داده‌های جدولی را بهبود داده و امکان مشاهده‌پذیری کامل (Observability) فرایند را در چارچوب MLOps فراهم کند. این چارچوب با هدف شناسایی، اصلاح و پایش خطاهای کیفیت داده طراحی شده و تلاش می‌کند شکاف میان مهندسی داده، مدل‌های زبانی بزرگ و سامانه‌های MLOps را کاهش دهد.

3.2 اهداف اختصاصی

- برای دستیابی به هدف اصلی، اهداف اختصاصی زیر دنبال می‌شوند:
۱. **شناسایی خودکار خطاهای کیفیت داده** شامل مقادیر گمشده (Missing Values)، داده‌های تکراری (Duplicates)، ناسازگاری‌های ساختاری (Schema Mismatch) و خطاهای معنایی (Semantic Errors)
 ۲. **طراحی و پیاده‌سازی موتور مسیریابی هیبرید (Hybrid Router)** به‌منظور تصمیم‌گیری هوشمند در تخصیص وظایف میان روش‌های مبتنی بر قواعد و مدل‌های زبانی بزرگ، با هدف کاهش هزینه محاسباتی و زمان پردازش.
 ۳. **پیاده‌سازی لایه استنتاج محلی مبتنی بر LLM** با استفاده از مدل‌های متن‌باز نظیر Mistral-7B و LLaMA-3-8B در قالب نسخه‌های کوانتیزه‌شده ۴ بیتی، بدون وابستگی به سرویس‌های ابری و API های تجاری.
 ۴. **طراحی معماری چندعامله (Multi-Agent)** شامل عامل تحلیل‌گر (Profiler Agent)، عامل پاکسازی (Cleaner Agent) و عامل ارزیاب (Reviewer Agent) برای مدیریت و اعتبارسنجی فرایند پاکسازی داده.
 ۵. **یکپارچه‌سازی سامانه با ابزارهای MLOps** به‌ویژه MLflow، به‌منظور ثبت، پایش و ردیابی معیارهای کیفیت داده، عملکرد مدل و اطلاعات مربوط به هر اجرای سیستم. (Run)
 ۶. **ارزیابی کمی عملکرد سیستم** از طریق مقایسه کیفیت داده پیش و پس از پردازش و اندازه‌گیری معیارهایی نظیر Completeness، Consistency، Duplicate Rate و Data Quality Improvement Score.
 ۷. **طراحی یک لایه پایش و مشاهده‌پذیری (Monitoring & Observability)** برای ردیابی تغییرات کیفیت داده، ثبت رخدادها، تحلیل عملکرد عامل‌ها و تشخیص Drift در داده‌های ورودی.
 ۸. **ارائه یک راهکار سبک‌وزن و قابل اجرا در محیط‌های دانشگاهی و سازمانی کوچک** که بدون نیاز به زیرساخت‌های پیچیده آموزش مدل یا سرویس‌های ابری، قابلیت بهره‌برداری عملی داشته باشد.

4. پیشینه پژوهش

4.1 پژوهش‌های مرتبط در حوزه پاکسازی داده با مدل‌های زبانی بزرگ

مقاله [۱] نشان می‌دهد که مدل‌های زبانی بزرگ (LLM) قابلیت بالایی در انجام وظایف مختلف آماده‌سازی داده دارند. این وظایف شامل پاکسازی داده، یکپارچه‌سازی داده و تطبیق ساختار یا Schema Matching است. نویسندگان کاربردهای LLM در حوزه داده را در سه دسته اصلی شامل پاکسازی داده (Data Cleaning)، یکپارچه‌سازی داده (Data Integration) و تبدیل داده (Data Transformation) طبقه‌بندی کرده‌اند.

مقاله [۲] در پژوهش LLMClean نشان دادند که مدل‌های زبانی قادرند قوانین پاکسازی داده را به‌صورت خودکار از خود داده‌ها استخراج کنند. این رویکرد که با عنوان "پاکسازی داده مبتنی بر زمینه Context-Aware Data Cleaning" شناخته می‌شود، در شناسایی ناسازگاری‌های معنایی عملکرد بهتری نسبت به روش‌های سنتی مبتنی بر قوانین ثابت دارد.

مقاله [۳] در پژوهش AutoDCWorkflow نشان دادند که مدل‌های زبانی بزرگ می‌توانند فرایندهای کامل پاکسازی داده را به‌صورت پویا و بدون نیاز به آموزش مجدد مدل تولید کنند. این رویکرد نقش مهمی در افزایش سطح خودکارسازی فرایندهای ETL ایفا می‌کند.

4.2 مبانی نظری در حوزه MLOps و مدیریت کیفیت داده

مقاله [۴] در پژوهش کلاسیک خود، مشکلات کیفیت داده را به دو دسته اصلی "خطاهای تک‌منبعی" و "خطاهای چندمنبعی" تقسیم کرده‌اند. این طبقه‌بندی همچنان یکی از مراجع اصلی در طراحی سامانه‌های اعتبارسنجی و پاکسازی داده محسوب می‌شود.

مقاله [۹] معماری MLOps را بر سه اصل اساسی شامل خودکارسازی (Automation)، پایش مستمر (Continuous Monitoring) و بازتولیدپذیری (Reproducibility) بنا کرده‌اند. در پروژه حاضر نیز از همین چارچوب برای طراحی لایه‌های ثبت رویدادها (Logging) و پایش (Monitoring) استفاده می‌شود.

علاوه بر این، در این پروژه از یک لایه اعتبارسنجی مبتنی بر قواعد (Rule-Based Validation Layer) برای بررسی کیفیت داده استفاده می‌شود. قواعد اعتبارسنجی با الهام از رویکردهای رایج در ابزارهایی نظیر Great Expectations طراحی شده‌اند و شامل بررسی مقادیر گم‌شده، یکتایی داده‌ها، سازگاری نوع داده و اعتبار فرمت‌ها هستند. همچنین MLflow به عنوان بستر اصلی ثبت، پایش و ردیابی اجرای آزمایش‌ها مورد استفاده قرار می‌گیرد.

4.3 معماری‌های عامل‌محور (Agent-Based) در مهندسی داده

مقاله [۵] نشان دادند که استفاده از عامل‌های مبتنی بر مدل‌های زبانی بزرگ (LLM Agents) در تشخیص خطاهای داده‌ای عملکرد بهتری نسبت به استفاده مستقیم از پرامپت‌های ساده دارد. در این پژوهش از الگوی ReAct (استدلال و اقدام) برای تصمیم‌گیری مرحله‌ای و حل مسائل پیچیده استفاده شده است.

مقاله [۷] در سامانه Data-Copilot نشان دادند که یک معماری چندعامله (Multi-Agent) قادر است کل جریان پردازش داده را مدیریت کند. با این حال، این سامانه وابستگی کامل به API مدل GPT-4 دارد و در محیط‌های ایزوله یا دارای محدودیت امنیتی قابل استفاده نیست.

مقاله [۸] با معرفی چارچوب DSPy نشان دادند که ترکیب ابزارهای برنامه‌نویسی مانند Python و Pandas با توان استدلال مدل‌های زبانی بزرگ، به‌ویژه در مسائل ساختاریافته، می‌تواند دقت و کیفیت خروجی را به‌طور محسوسی افزایش دهد.

4.4 خلأهای پژوهشی و جایگاه پروژه پیشنهادی

بررسی پژوهش‌های موجود نشان می‌دهد که با وجود پیشرفت‌های اخیر در استفاده از مدل‌های زبانی بزرگ برای پاکسازی و مدیریت داده، همچنان چندین خلأ مهم وجود دارد. بسیاری از راهکارهای موجود به سرویس‌های ابری وابسته‌اند، از قابلیت پایش و مشاهده‌پذیری در چارچوب MLOps برخوردار نیستند و یا توانایی درک معنایی داده‌ها را به‌صورت کامل ارائه نمی‌دهند.

جدول زیر مهم‌ترین خلأهای موجود و راهکار ارائه‌شده در این پروژه را نشان می‌دهد:

پاسخ این پروژه	خلأ	روش/پژوهش موجود
اجرای کامل محلی و استفاده از MLflow برای ثبت و پایش	وابستگی به سرویس‌های ابری و فقدان قابلیت پایش MLOps	LLMClean [2]
پیاده‌سازی لایه Logging و Monitoring همراه با تشخیص Drift	نبود قابلیت مشاهده‌پذیری و پایش تغییرات داده	AutoDCWorkflow [3]
ترکیب اعتبارسنجی مبتنی بر قانون با تحلیل معنایی مبتنی بر LLM	عدم توانایی در درک معنایی داده‌ها	Great Expectations [6]
استفاده از مدل‌های متن‌باز محلی مانند Mistral و Llama بدون هزینه API	وابستگی به GPT-4 و هزینه بالای استفاده	Data-Copilot [7]

5. نوآوری و ادعای برتری (Contribution)

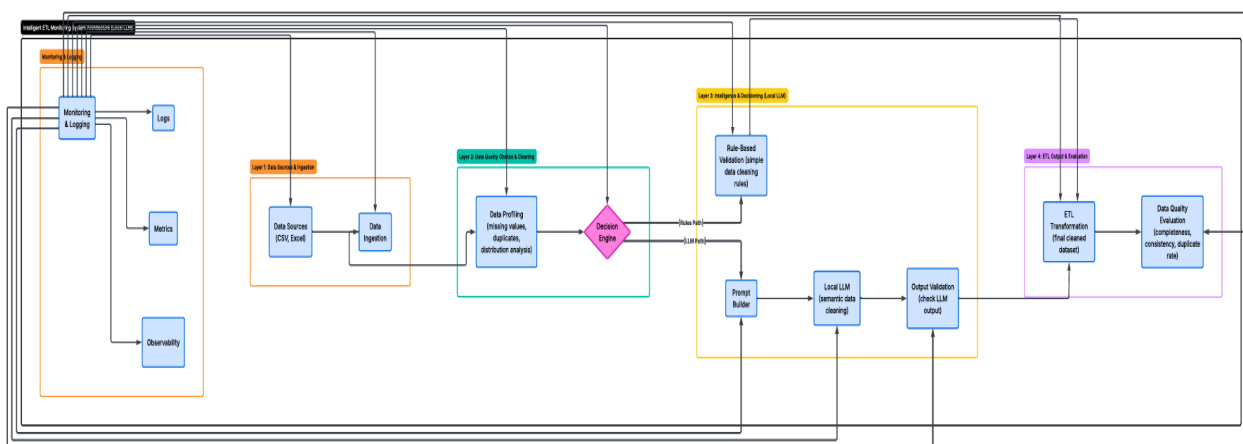
ادعای برتری این پروژه نسبت به روش‌های موجود بر سه پایه استوار است:

- رویکرد هیبریدی (Hybrid Routing):** برخلاف سیستم‌هایی که تمام داده‌ها را به LLM می‌فرستند، سیستم ما ابتدا خطاهای ساده را با روش‌های Rule-based (با هزینه محاسباتی $O(n)O(n)O(n)$) حل کرده و تنها داده‌های نیازمند استدلال معنایی را به LLM می‌سپارد. این کار باعث کاهش شدید تاخیر (Latency) و هزینه محاسباتی می‌شود.
- حفظ حریم خصوصی و اجرای محلی:** استفاده از مدل‌های Local (مانند Quantized Mistral یا LLaMA) وابستگی به سرویس‌های ابری خارجی را حذف کرده و امنیت داده‌های حساس جداول را تضمین می‌کند.

- یکپارچگی با **MLOps**: بر خلاف ابزارهای صرفاً تحقیقاتی، این پروژه دارای لایه مشاهده‌پذیری (Observability) است که معیارهای کیفیت داده (Data Quality) و عملکرد مدل (Drift/Latency) را به صورت لحظه‌ای پایش می‌کند.

6. معماری سیستم

سیستم پیشنهادی از یک معماری چندلایه تشکیل شده است که شامل لایه‌های پردازش داده، تحلیل، تصمیم‌گیری، مدل زبانی، و پایش است.



6.1 لایه دریافت داده (Data Ingestion Layer)

داده‌ها از منابعی مانند فایل‌های CSV یا Excel وارد سیستم می‌شوند. وظایف این لایه خواندن داده، تشخیص ساختار اولیه و استانداردسازی اولیه داده است.

6.2 لایه پروفایلینگ داده (Data Profiling Layer)

در این مرحله، تحلیل اولیه بر روی داده انجام می‌شود که تشخیص Missing Values، تشخیص Duplicate Records، بررسی نوع داده‌ها و تحلیل توزیع داده‌ها انجام خواهد شد. خروجی این لایه یک گزارش کیفیت اولیه داده است.

6.3 لایه اعتبارسنجی مبتنی بر قوانین (Rule-Based Validation Layer)

در این لایه مجموعه‌ای از قوانین ساده اجرا می‌شود که شامل عدم وجود مقدار Null، یکتایی داده‌ها (Unique Constraint) و بررسی فرمت‌ها (Regex Validation) است. هدف این لایه کاهش حجم داده‌های نیازمند تحلیل پیچیده است.

6.4 لایه تصمیم‌گیری (Decision Engine)

این بخش نقش مغز کنترل سیستم را دارد. عملکرد آن به صورت تعیین نوع خطا (ساده یا معنایی)، هدایت داده به Rule-Based یا LLM و مدیریت جریان پردازش است. این لایه باعث کاهش هزینه محاسباتی مدل زبانی می‌شود.

6.5 لایه مدل زبانی بزرگ محلی (Local LLM Layer).

ویژگی‌های این بخش استفاده از مدل‌های متن‌باز مانند Mistral یا LLaMA، اجرای محلی و پردازش داده‌های دارای خطای معنایی است. وظایف آن نیز اصلاح خطاهای معنایی (Semantic Correction)، تطبیق نام ستون‌ها (Schema Matching)، پیشنهاد مقدار اصلاح‌شده و تفسیر داده‌های غیرساخت‌یافته است.

6.6 اعتبارسنجی خروجی

نتایج تولیدشده توسط مدل بررسی شده و در صورت تأیید، اعمال می‌شوند.

6.7 لایه ETL Transformation Layer

در این مرحله خروجی Rule-Based و LLM ترکیب می‌شود و داده نهایی پاکسازی می‌شود و تبدیل ساختاری انجام می‌شود.

6.8 لایه پایش و لاگ (Logging Layer & Monitoring)

این لایه مسئول مشاهده‌پذیری سیستم است. قابلیت‌های آن ثبت تمام عملیات ETL، ثبت ورودی و خروجی LLM، ثبت خطاها و اصلاحات و ایجاد تاریخچه پردازش داده است.

6.9 لایه ارزیابی کیفیت داده (Data Quality Engine)

در این مرحله کیفیت داده قبل و بعد از پردازش مقایسه می‌شود که موارد زیر مورد نظر است:

- Completeness
- Consistency
- Duplicate Rate
- Accuracy Improvement

7. زیرساخت عامل‌محور (Agentic Infrastructure)

7.1 رویکرد معماری

در این پروژه، مدل زبانی بزرگ (LLM) صرفاً به عنوان یک تابع تولید متن مورد استفاده قرار نمی‌گیرد، بلکه از معماری عامل‌محور مبتنی بر الگوی **ReAct (Reasoning and Acting)** بهره گرفته می‌شود. این رویکرد به عامل‌ها امکان می‌دهد تا ضمن استدلال درباره مسئله، ابزارهای مناسب را انتخاب کرده و تصمیم‌گیری‌های

چند مرحله‌ای انجام دهند. در نتیجه، فرآیند پاک‌سازی و اعتبارسنجی داده‌ها به‌صورت هوشمند، قابل‌ردیابی و قابل‌توضیح اجرا می‌شود.

7.2 عامل‌های اصلی سیستم

عامل پروفایلر (Profiler Agent)

مسئولیت: تحلیل اولیه داده‌ها و تولید گزارش کیفیت داده

وظایف اصلی این عامل عبارت‌اند از:

- محاسبه درصد مقادیر مفقود (Missing Values) برای هر ستون؛
- شناسایی رکوردهای تکراری از طریق روش‌های تطابق دقیق (Exact Duplicate) و تطابق تقریبی (Fuzzy Duplicate)؛
- محاسبه آمار توصیفی شامل حداقل، حداکثر، میانگین و انحراف معیار؛
- شناسایی داده‌های پرت (Outliers)؛
- تولید گزارش JSON شامل شاخص‌های کیفیت داده و امتیاز اولیه کیفیت جهت استفاده در موتور تصمیم‌گیری.

عامل پاک‌سازی (Cleaner Agent)

مسئولیت: دریافت گزارش Profiler Agent و اعمال اصلاحات معنایی بر داده‌ها

وظایف اصلی این عامل عبارت‌اند از:

- تطبیق و یکپارچه‌سازی ستون‌های ناهمنام از طریق Schema Matching؛
- استفاده از تکنیک Few-Shot Prompting مبتنی بر نمونه‌های تخصصی دامنه مسئله؛
- پیشنهاد مقادیر اصلاح‌شده برای ناسازگاری‌های معنایی و خطاهای داده‌ای؛
- اجرای مدل‌های زبانی محلی از طریق Ollama، شامل Mistral-7B-Instruct یا LLaMA-3-8B با قالب فشرده GGUF چهار بیتی.

عامل ناظر (Reviewer Agent)

مسئولیت: ارزیابی و اعتبارسنجی خروجی تولیدشده توسط Cleaner Agent

وظایف اصلی این عامل عبارت‌اند از:

- مقایسه خروجی با قوانین مرجع (Ground Rules) به‌منظور جلوگیری از بروز Hallucination؛
- پذیرش و اعمال تنها تغییراتی که دارای امتیاز اطمینان (Confidence Score) بالاتر از آستانه تعیین‌شده باشند؛
- ثبت تمامی تصمیمات و اصلاحات در MLflow به‌منظور ممیزی، ردیابی و تحلیل عملکرد سیستم.

7.3 جریان تعامل عامل‌ها

فرآیند اجرای سیستم به‌صورت زیر انجام می‌شود:

۱. Profiler Agent داده‌ها را تحلیل کرده و گزارش کیفیت را در قالب JSON تولید می‌کند.
۲. Decision Engine براساس نتایج گزارش، مسیر پردازش مناسب را انتخاب می‌کند.
۳. در صورت نیاز به تحلیل معنایی، Cleaner Agent فعال شده و اصلاحات پیشنهادی را تولید می‌کند.
۴. Reviewer Agent خروجی را اعتبارسنجی کرده و تغییرات معتبر را اعمال می‌کند.
۵. تمامی مراحل، تصمیمات و نتایج در MLflow ثبت و ذخیره می‌شوند.

این معماری عامل‌محور به صورت ماژولار در قالب Agent های مستقل پیاده‌سازی شده است. طراحی سیستم با الهام از الگوی ReAct انجام شده و امکان توسعه آینده با چارچوب‌هایی نظیر LangChain یا CrewAI را نیز فراهم می‌کند. در نسخه فعلی، هماهنگی میان عامل‌ها توسط لایه Orchestrator و موتور تصمیم‌گیری داخلی انجام می‌شود.

8. زیرساخت MLOps

8.1 رویکرد کلی

در این پروژه، **MLflow** به عنوان بستر اصلی پایش، ردیابی و مشاهده‌پذیری (Observability) مورد استفاده قرار می‌گیرد. تمامی فعالیت‌ها در قالب Run ثبت شده و امکان مقایسه، تحلیل و بازتولید نتایج را فراهم می‌کنند.

8.2 اطلاعات ثبت‌شده در هر Run

هدف	متغیرهای ثبت‌شده	دسته لاگ
مقایسه قبل/بعد پردازش	Completeness, Duplicate Rate, Consistency	Data Quality
پایش کارایی مدل	Latency per row, Confidence Score	LLM Performance
شناسایی تغییر در داده ورودی	توزیع آماری ستون‌ها بین Batch ها	Drift Detection
Auditability و Debug	نوع خطا، مسیر انتخابی (Rule/LLM)، اصلاح اعمال‌شده	Agent Actions
ذخیره خروجی‌های قابل بازبینی	گزارش HTML ، JSON کیفیت داده	Artifacts

8.3 تشخیص رانش داده (Data Drift)

رانش داده یکی از مهم‌ترین چالش‌های سامانه‌های مبتنی بر یادگیری ماشین است. در معماری پیشنهادی، تشخیص رانش در لایه ETL و پیش از ورود داده‌ها به مدل انجام می‌شود. این فرآیند شامل موارد زیر است:

- مقایسه توزیع آماری ستون‌های عددی بین Batch جاری و Batch مرجع با استفاده از معیارهایی نظیر KL Divergence و آزمون Kolmogorov-Smirnov؛
- ثبت هشدار و تولید گزارش در MLflow در صورت شناسایی رانش داده؛
- تشخیص زودهنگام تغییرات داده‌های ورودی که نسبت به شناسایی رانش پس از استقرار مدل، سریع‌تر، کم‌هزینه‌تر و مؤثرتر است.

8.4 استقرار و اجرای MLflow

MLflow به‌صورت کاملاً محلی (Local) اجرا می‌شود و نیازی به زیرساخت ابری ندارد. رابط کاربری آن از طریق نشانی localhost:5000 در دسترس است. در هر Run، پارامترها، معیارهای ارزیابی و Artifacts ثبت می‌شوند تا امکان ردیابی، تحلیل و بازتولید کامل نتایج فراهم شود.

9. دیتاست‌ها

برای ارزیابی سیستم از دیتاست‌های دانشگاهی معتبر با ویژگی‌های متنوع استفاده می‌شود. تمام دیتاست‌ها عمومی و قابل دسترس هستند:

نوع خطای اصلی	ستون	ردیف	منبع	دیتاست
Missing Values, Type Mismatch	12	~891	Kaggle / UCI	Titanic
Inconsistency, Noise, Duplicates	15	~48,842	UCI ML Repository	Adult Income
Schema Mismatch, Semantic Error	متنوع	متنوع	OpenML.org	OpenML Dirty Tabular
Semantic Errors, Mixed Formats	10	~1000	تولیدشده [2]	دیتاست مصنوعی (LLMClean)

10. معیارهای ارزیابی

ارزیابی سیستم در سه سطح انجام می‌شود: (۱) کیفیت داده، (۲) کارایی مدل، (۳) پایش و مشاهده‌پذیری. جدول زیر معیارها، روش اندازه‌گیری و آستانه موفقیت را نشان می‌دهد:

مرجع	آستانه موفقیت	روش اندازه‌گیری	معیار	هدف ارزیابی
[2],[3]	$\geq 95\%$	ETL قبل/بعد	Completeness (%)	کیفیت داده
[4]	$\geq 90\%$	Rule Validation	Consistency Score	کیفیت داده

کیفیت داده	Duplicate Rate (%)	Before/After	$\leq 1\%$	[1],[2]
کارایی مدل	LLM Latency (s)	Per-row timing	$< 5s/row$	[5]
کارایی مدل	Error Correction Rate	Manual Annotation	$\geq 80\%$	[1],[5]
پایش	Log Coverage (%)	Log Audit	100%	[6],[11]
پایش	Drift Detection Delay	Simulation Test	$< 1 \text{ batch}$	[9]

10.1 فرمول‌های محاسباتی معیارهای اصلی

۱. کامل بودن داده‌ها: (Completeness)

$$\text{Completeness} = \frac{\text{تعداد مقادیر غیر تهی (non-null)}}{\text{کل مقادیر}} \times 100$$

۲. نرخ داده‌های تکراری: (Duplicate Rate)

$$\text{Duplicate Rate} = \frac{\text{تعداد رکوردهای تکراری}}{\text{کل رکوردها}} \times 100$$

۳. نرخ اصلاح خطا: (Error Correction Rate)

$$\text{Error Correction Rate} = \frac{\text{تعداد خطاهای اصلاح‌شده توسط LLM}}{\text{کل خطاهای معنایی شناسایی‌شده}} \times 100$$

۴. امتیاز بهبود کیفیت داده: (Data Quality Improvement Score)

$$\text{DQIS} = \frac{\text{امتیاز پیش از پردازش} - \text{امتیاز پس از پردازش}}{\text{امتیاز پیش از پردازش}} \times 100$$

11. فناوری‌های مورد استفاده

توضیح	نقش	ابزار / کتابخانه	لایه
-------	-----	------------------	------

پردازش داده	Python, Pandas, NumPy	ETL Core	پردازش داده‌های جدولی، تحلیل داده و اجرای قواعد اعتبارسنجی
مدل زبانی بزرگ	Mistral-7B / LLaMA-3-8B (4-bit)	Local Inference	اجرای مدل‌های زبانی به صورت محلی بدون وابستگی به API های خارجی
سرویس مدل	Ollama / vLLM	Inference Server	مدیریت و ارائه سرویس مدل‌های زبانی از طریق API محلی
چارچوب عامل‌محور	Orchestrator داخلی مبتنی بر Python	Agentic Logic	پیاده‌سازی هماهنگی میان عامل‌ها، مدیریت جریان پردازش، کنترل تصمیم‌گیری و اجرای وظایف چندمرحله‌ای در معماری Agentic سیستم.
ثبت و پایش	MLflow + Python logging	Observability	ثبت معیارها، گزارش‌گیری و ردیابی آزمایش‌ها
اعتبارسنجی داده	Great Expectations	Data Validation	تعریف قواعد اعتبارسنجی و ارزیابی کیفیت داده
محیط توسعه	VS Code, Jupyter, Git	Dev Environment	توسعه، آزمایش و نسخه‌بندی

12. نیازمندی‌های زیرساختی

12.1 نیازمندی‌های سخت‌افزاری

جدول زیر حداقل منابع موردنیاز برای اجرای سامانه پیشنهادی را نشان می‌دهد.

منبع	مشخصات	توضیح فنی
GPU اصلی	16–24 GB VRAM (RTX 3090/4090)	اجرای مدل‌های ۷ B با کوانتیزیشن ۴ بیتی (GGUF)

گزینه رایگان برای محیط دانشگاهی	Google Colab Pro / Kaggle T4 16GB	GPU جایگزین
بارگذاری دیتاست‌های جدولی و-Pandas in-memory	حداقل ۱۶ گیگابایت (۳۲ گیگابایت توصیه می‌شود)	RAM
Rule-based validation و پردازش‌های موازی Pandas	8 هسته یا بیشتر	CPU
وزن‌های مدل + (4-7GB)~ لاگ‌ها و دیتاست‌ها	حداقل 50 GB SSD	نخیره‌سازی

12.2 برآورد منابع محاسباتی

نیازمندی حافظه گرافیکی (VRAM)

مدل **Mistral-7B** با کوانتیزیشن چهار بیتی (**GGUF Q4_K_M**) تقریباً به ۴٫۵ گیگابایت حافظه گرافیکی نیاز دارد. برآورد مصرف حافظه به شرح زیر است:

- پارامترهای مدل: حدود ۷ میلیارد پارامتر $\times$ ۴ بیت $\div$ ۸ $\approx$ ۳٫۵ گیگابایت؛
- حافظه **KV Cache** برای طول دنباله ۲۰۴۸ توکن: حدود ۰٫۵ گیگابایت؛
- سربار زمان اجرا (**Runtime Overhead**) مربوط به **Ollama**: حدود ۰٫۵ گیگابایت؛
- مجموع مصرف حافظه: حدود ۴٫۵ گیگابایت.

بنابراین اجرای مدل روی کارت‌های گرافیکی نظیر **NVIDIA T4** با ۱۶ گیگابایت حافظه یا **RTX 3090** بدون محدودیت امکان‌پذیر است. استفاده از کوانتیزیشن چهار بیتی نسبت به نمایش **FP16**، مصرف حافظه را حدود ۷۵ درصد کاهش می‌دهد، در حالی که افت دقت معمولاً کمتر از ۲ درصد باقی می‌ماند.

نیازمندی حافظه اصلی (RAM)

حافظه **RAM** عمدتاً برای پردازش داده‌های جدولی توسط **Pandas** و اجرای سرویس‌های جانبی مورد استفاده قرار می‌گیرد.

- مجموعه داده **Titanic** (۸۹۱ ردیف و ۱۲ ستون): حدود ۵۰ کیلوبایت؛
- مجموعه داده **Adult Income** (۴۸٫۸۴۲ ردیف و ۱۵ ستون): حدود ۵ مگابایت؛
- مجموعه داده‌های بزرگ **OpenML Dirty** (تا ۵۰۰ هزار ردیف): حدود ۵۰۰ مگابایت؛
- سرویس **MLflow** و محیط اجرایی **Python**: حدود ۱ تا ۲ گیگابایت؛
- حافظه موقت (**Buffer**) و سربار سیستم: حدود ۲ تا ۴ گیگابایت.

بر این اساس، ۱۶ گیگابایت **RAM** برای اجرای سامانه کافی است؛ با این حال، استفاده از ۳۲ گیگابایت حافظه برای اجرای هم‌زمان چند پردازش و افزایش پایداری سیستم توصیه می‌شود.

نیازمندی پردازنده (CPU)

بار پردازشی CPU عمدتاً مربوط به اعتبارسنجی‌های مبتنی بر قواعد و پردازش‌های داده‌ای است و پیچیدگی زمانی آن تقریباً خطی ($O(n)$) نسبت به تعداد رکوردها است.

- پردازش ۵۰۰ هزار رکورد با استفاده از **Great Expectations** روی پردازنده ۸ هسته‌ای: حدود ۳۰ تا ۶۰ ثانیه؛
- تولید گزارش کیفیت داده توسط **Profiler Agent**: حدود ۵ تا ۱۰ ثانیه؛
- ثبت لاگ‌ها در **MLflow**: دارای سربرار ناچیز و عمدتاً وابسته به سرعت دیسک؛
- اجرای مدل زبانی بر عهده **GPU** بوده و **CPU** تنها عملیات **Rule-Based** و هماهنگی عامل‌ها را مدیریت می‌کند.

12.3 زیرساخت نرم‌افزاری

سامانه پیشنهادی بر پایه ابزارها و فناوری‌های زیر پیاده‌سازی می‌شود:

- **Ollama** برای سرویس‌دهی محلی مدل‌های زبانی از طریق **REST API** بر روی پورت **localhost:11434**؛
- **MLflow** برای ردیابی آزمایش‌ها، پایش عملکرد و ذخیره **Artifacts** از طریق رابط کاربری **localhost:5000**؛
- **Python 3.10** یا بالاتر به عنوان زبان اصلی توسعه؛
- **CUDA 12.0** یا بالاتر برای بهره‌برداری از شتاب‌دهنده‌های گرافیکی **NVIDIA**؛
- **Git** برای مدیریت نسخه، همکاری تیمی و کنترل تغییرات کد.

13. جمع‌بندی

در این پروژه، یک چارچوب سبک برای پایش و پاکسازی داده‌های **ETL** ارائه شد که در آن از ترکیب روش‌های مبتنی بر قوانین و مدل‌های زبانی بزرگ استفاده شده است. این رویکرد امکان بهبود کیفیت داده و افزایش شفافیت در فرآیند **ETL** را فراهم می‌کند و می‌تواند به عنوان یک راهکار قابل اجرا در محیط‌های آموزشی مورد استفاده قرار گیرد.

14. منابع

- [1]Zhang et al., "A Survey of LLM-based Data Preparation", 2024 .
- [2]Li et al., "LLMClean: Context-Aware Data Cleaning", 2024 .
- [3]Wang et al., "AutoDCWorkflow", 2024 .
- [4]Rahm, E., Do, H., "Data Cleaning: Problems and Current Approaches ."

- [5]Chen et al., "LLM Agents for Tabular Data Cleaning", 2025 .
- [6]Great Expectations Documentation .
- [7] Zhang, W., et al. (2023). "Data-Copilot: Bridging Billions of Data and Humans with Autonomous Workflow."
- [8] Khattab, O., et al. (2024). "DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines." *International Conference on Learning Representations (ICLR)*.
- [9] Kreuzberger, C., et al. (2023). "Machine Learning Operations (MLOps): Overview, Definition, and Architecture." *IEEE Access*.
- [10] Chen, Y., & Wang, H. (2025). "LLMOps: Operationalizing Large Language Models in Enterprise and Air-gapped Environments." *Journal of Data and Information Quality (JDIQ)*.
- [11] MLflow Team. (2024). MLflow Documentation.
<https://mlflow.org/docs/latest/index.html>
- [12] Ollama. (2024). Ollama — Run LLMs Locally. <https://ollama.ai>